

PRISM → World Monitor Scale: Complete Change List & Roadmap

Reality check first

World Monitor is not a hackathon project scaled up — it's a mature product: 2M+ users, 56 map layers, live satellite/AIS/military feeds, a real backend team, and it's been running long enough to get press (WIRED). Getting PRISM there isn't one big rebuild, it's **6-8 sequential phases**, most of which are solo-buildable if you sequence them right and lean on free/public data instead of building infrastructure from scratch.

Everywhere something looks genuinely out of reach for a solo student, I've flagged it and given you the actual workaround — free-tier equivalents, phased scope cuts, or "do this instead."

1. Data Infrastructure — the biggest gap

Where you are: Sandbox cache, auto-fallback to local memory, likely single-run/manual article fetches.

Where World Monitor is: Continuous ingestion from ACLED, UCDP, NASA FIRMS, USGS, OpenSky, AISStream, FRED, IMF, BIS, + 500 RSS feeds — all running on a scheduler, 24/7.

What needs to change

- Move from "fetch on demand" to a **scheduled ingestion pipeline** (cron jobs / background workers pulling every N minutes)
- Add a **real database** (Postgres or similar) — sandbox/local-memory cannot survive concurrent users or restarts
- Build a **deduplication layer** — same story from 5 outlets shouldn't create 5 entries
- Add a **source registry** — structured metadata per outlet (bias rating, reliability history, region) instead of ad hoc tagging

If this feels out of reach solo

- You don't need satellite/AIS/military data to start. World Monitor's *news* layer alone (RSS + bias scoring) is your MVP — that's literally what PRISM already does. Free RSS feeds (Reuters, AP, BBC, Al Jazeera, etc.) all have public feeds — no scraping needed.
 - Use **Supabase or Firebase Firestore** (free tier) instead of managing your own Postgres server — near-zero ops overhead for a solo dev.
 - Skip AIS ships/military tracking entirely for v1. That's a *layer*, not the product. World Monitor started as one thing and added layers over years.
-

2. Trust & Data Integrity (fix before scaling — non-negotiable)

This was flagged in your screenshot: the demo article ("Sovereign Election Transparency Protocol," "0.00% data deviation") reads like generated placeholder content, not a real wire story.

What needs to change

- Every "Verified by Reuters/AP" tag must link to a real, fetchable article URL. No exceptions once real users see this.
- Add a **provenance check step** in your agent pipeline: before publishing a verdict, confirm the source article actually exists and was actually fetched (not hallucinated by the LLM).
- Log raw source text alongside the AI's summary, so you can audit for drift.

Why this is priority zero: a "transparency AI" tool that turns out to fabricate its own sources is not a bug, it's the whole business proposition failing. This must be solid before any scale conversation matters.

3. AI / Multi-Agent Architecture — cost and scale

Where you are: 3 analyst agents + moderator, likely running on-demand per query (based on hackathon architecture).

Where World Monitor is: AI-synthesized briefings running continuously across thousands of stories/day.

What needs to change

- **Batch processing, not per-request** — running your full multi-agent debate on every single article live is expensive and slow. Instead: ingest → queue → process in batches on a schedule.
- **Model tiering** — use a cheap/fast model (e.g., a smaller Claude or open-weight model via Hugging Face) for initial triage/filtering, and reserve your full 3-agent debate for stories that clear a relevance threshold. Running full multi-agent debate on every RSS item globally will bankrupt you fast at scale.
- **Caching** — if the same story is asked about by 100 users, you compute the verdict once, not 100 times.

If cost is the blocker

- Start with a **daily story cap** (e.g., top 50 trending stories get full multi-agent treatment; everything else gets a lighter single-pass check). This mirrors how real newsrooms triage.
 - Use **Hugging Face open-source models** for the cheap first-pass layer — free/near-free inference for triage, save paid API calls for the real debate step.
-

4. Frontend — feed view vs. global map view

Where you are: List-based "Transparency Feed" (card layout, category tabs).

Where World Monitor is: A 3D interactive global map with toggleable layers, zoom, time range.

What needs to change

- This is a **genuinely large frontend lift** — a real-time WebGL/3D globe (World Monitor likely uses something like Mapbox GL, deck.gl, or a custom Three.js globe) is a different skillset than a card-based feed.
- Layer toggle system (conflicts, weather, economic, etc.) needs a **layer registry pattern** — each layer is its own data source + its own map rendering logic.

Realistic path

- **You don't need a 3D globe for v1.** A 2D map (Leaflet.js or Mapbox, both have generous free tiers) with pins/heatmap for story locations gets you 80% of the "global monitor" feel for a fraction of the build time.
 - Keep your current feed view as a **secondary "list mode"** — many users actually prefer it over the map for reading. World Monitor itself still has panel-based views alongside the map.
 - If you want the 3D globe specifically, `react-globe.gl` is a free open-source library that gets you most of the way without building WebGL from scratch.
-

5. Infrastructure & Hosting

Where you are: Firebase hosting (based on your `.web.app` domain) — fine for a static/light app.

Where World Monitor needs: Handling live data streams, WebSocket updates, background jobs, and traffic at scale.

What needs to change

- Firebase Hosting alone won't run scheduled background jobs — you'll need either **Firebase Cloud Functions** (scheduled functions) or a small always-on server (Railway, Render, or Fly.io all have free/cheap tiers good for solo projects).
- Add a **CDN + caching layer** (Cloudflare, free tier) once you have real traffic — this is exactly what the cloud icon in your original logo set represents, and it directly reduces your hosting costs at scale.

If this feels like too much infra to manage solo

- **Railway or Render free/hobby tier** can run your scheduled scraper + agent pipeline without you managing servers.

- Don't provision for 2M users on day one. World Monitor didn't start at that scale either — build for hundreds, not millions, and scale infra when you actually hit those numbers.

6. Monetization Layer

Where World Monitor is: Free map + Pro tier (AI analyst, personalized digests, MCP for Claude/GPT).

What needs to change

- Add **auth + subscription billing** — Stripe is the standard here, has a solid free-to-start integration.
- Decide your **free vs. Pro split** now, before scaling: e.g., free = limited daily "Verify a claim" queries + public feed; Pro = unlimited verification, deeper agent transparency logs, saved history.
- **MCP server** (letting Claude/GPT query PRISM directly) is a genuinely achievable differentiator for you specifically — Anthropic's MCP spec is open, and you could expose "verify this claim" as an MCP tool. This is a real edge over most bias-checking tools, since almost none of them are agent-queryable yet.

7. Team / Workflow Reality

World Monitor's founder (per the WIRED piece) had real engineering resources behind it. You're solo, second-year, alongside coursework and an IEEE paper. Be honest about sequencing:

Phase	Focus	Rough timeframe (solo, part-time)
0	Fix trust/provenance issues in current build	1-2 weeks
1	Real scheduled ingestion + database (no map yet)	3-4 weeks
2	Cost-efficient agent pipeline (batching, tiering)	2-3 weeks
3	2D map view (Leaflet/Mapbox) replacing/supplementing feed	3-4 weeks
4	Auth + Stripe + free/Pro split	2 weeks
5	MCP server for agent-queryable verification	1-2 weeks
6	Only after real users exist: 3D globe, more layers, more data sources	Ongoing

Do not build phases 3+ before phase 0-2 are solid. A pretty globe on top of unreliable data is worse than a plain list on top of trustworthy data — trust is your actual product.

Summary: what's genuinely hard vs. what's just work

Genuinely hard for solo/free-tier	Totally doable solo with free tools
Real-time AIS ship tracking, military base data, satellite fire detection (these need paid/gov data partnerships World Monitor likely built over time)	RSS ingestion, bias scoring, multi-agent verification (you already have this)
2M-user infra day one	Hundreds-to-thousands of users on free/hobby-tier hosting
Full 56-layer map system	1 solid layer (news/bias) done well, expand later
3D WebGL globe from scratch	<code>react-globe.gl</code> or a 2D Leaflet map

The honest path: **you don't need to match World Monitor's breadth. You need to out-execute it on one dimension — transparency/multi-agent bias verification — where you already have a real head start.** Layers and 3D globes are additive later; they're not what makes the business work.

Want next: a concrete Phase 0-2 build checklist with specific libraries/APIs to wire up, or a pricing model worked out for the free/Pro split?